

UKB GCTA GREML Workflow Handoff

Last updated: 2026-05-28

This document is a transfer-of-knowledge guide for the UK Biobank blood-trait SNP heritability workflow in this repository. It is written for the next person who needs to rerun, audit, or extend the analysis after the original workflow owner leaves the project.

The workflow prepares trait-specific GCTA input files from UKB phenotype and covariate tables, builds a genomic relationship matrix (GRM), runs GCTA GREML for each blood trait, and summarizes heritability outputs.

Primary traits currently configured:

```
* '30000'  
* '30010'  
* '30020'  
* '30040'  
* '30050'  
* '30060'  
* '30080'  
* '30120'  
* '30130'  
* '30140'
```

Primary handoff products:

```
gcta_inputs/trait_manifest.tsv  
gcta_inputs/traits_to_run.txt  
gcta_inputs/traits/<trait>/<trait>.phen  
gcta_inputs/traits/<trait>/<trait>.covar  
gcta_inputs/traits/<trait>/<trait>.qcovar  
gcta_inputs/traits/<trait>/<trait>.keep  
grm/blood_grm.grm.bin  
grm/blood_grm.grm.N.bin  
grm/blood_grm.grm.id  
results/trait_<trait>.hsq  
results/hsq_summary.tsv
```

0. Simplified Order Of Operations

Start here and do the steps in this order.

This is the short rerun recipe for the current UKB RAP setup.

1. Confirm you are on UKB RAP with 'dx' available.
2. Go to the cloned repository directory.
3. Run Step 1 input preparation.
4. Check the generated per-trait '.phen/.covar/.qcovar/.keep' files.
5. Build a GRM (typically on an unrelated/QC subset using '--keep').
6. Run GREML one trait at a time (or loop all traits).
7. Summarize all '.hsq' files into one TSV.
8. Verify trait-level sample sizes, h2 values, and missing/failed traits.
9. Archive command lines, software versions, and key output files.

Minimal commands (local repo execution pattern):

```

Rscript scripts/prepare_gcta_inputs.R \
--fam "/mnt/project/Bulk/Genotype Results/Genotype calls/ukb22418_c1_b0_v2.fam" \
--pheno-csv "/mnt/project/Sool/GCTA_Heritability/data/phenotype/raw_blood_phenotypes.csv" \
--covar-csv "/mnt/project/Sool/covariates_processed_all.tsv" \
--covar-id-column IID \
--covar-preset ukb_blood_lab_shared \
--trait-codes 30000 30010 30020 30040 30050 30060 30080 30120 30130 30140 \
--num-pcs 20 \
--outdir gcta_inputs

bash scripts/run_gcta_make_grm.sh \
--gcta "scripts/gcta_docker.sh" \
--mbfile examples/genotype_prefixes.mbfile \
--threads 16 \
--out grm/blood_grm

bash scripts/run_gcta_greml_all_traits.sh \
--gcta "scripts/gcta_docker.sh" \
--grm grm/blood_grm \
--traits-file gcta_inputs/traits_to_run.txt \
--input-dir gcta_inputs/traits \
--threads 16 \
--out-dir results

python3 scripts/summarize_hsqa.py \
--results-dir results \
--out results/hsqa_summary.tsv

```

0A. Quick FAQ

If you want to know what this workflow estimates, read 'Scientific Question'.

If you want to know the exact input files, read 'Detailed Input File Inventory'.

If you want to know how trait-specific files are built, read 'Main Workflow Logic', Step 1.

If you want to know how covariates are selected, read 'Parameter Table And Meaning' and 'Main Workflow Logic', Step 1.

If you want to know how GRM construction is handled at UKB scale, read 'Main Workflow Logic', Step 2, and 'Known Caveats'.

If you want to run jobs as RAP Swiss Army Knife jobs instead of interactive sessions, read 'How To Run The Workflow'.

If you want expected outputs for handoff, read 'Expected Output Directory Layout' and 'Minimal Handoff Checklist'.

If you want common failure triage, read 'Failure Modes And How To Debug'.

1. Scientific Question

The central question is:

How much phenotypic variance in selected UKB blood traits is explained by additive common-SNP effects captured by a genotype-derived relationship matrix?

Operationally, each trait is modeled with GCTA REML:

$$h^2_{\text{GREML}} = V(G) / V_p$$

where 'V(G)' is additive genetic variance from the GRM and 'Vp' is total phenotypic variance after covariate adjustment.

2. Previous Research And What This Workflow Mimics

This workflow follows the standard GCTA GREML framing used in UKB-scale heritability analyses:

- * Build phenotype and covariate inputs aligned to PLINK sample IDs.
- * Build a GRM from genotype data.
- * Fit one REML model per trait using covariates and quantitative covariates.
- * Extract and summarize 'V(G)/Vp' with uncertainty and likelihood metrics.

It also follows UKB operational constraints on RAP:

- * Heavy compute is expected to run via 'dx run' jobs.
- * Protected data remains in RAP project storage.
- * One trait per GREML invocation is preferred for fault tolerance and reruns.

3. Repository Layout

```
UKB_GCTA_Heritability/  
  README.md  
  dnanexus/  
    runbook_swiss_army_knife.md  
  examples/  
    genotype_prefixes.mbfile  
  traits/  
    blood_trait_codes.txt  
  scripts/  
    prepare_gcta_inputs.R  
    run_gcta_make_grm.sh  
    run_gcta_greml_one_trait.sh  
    run_gcta_greml_all_traits.sh  
    summarize_hsq.py  
    gcta_docker.sh  
    submit_step1_prepare_dx.sh  
    submit_step1_prepare_dx.R  
    submit_step2_grm_dx.R
```

4. Main RAP Paths

Typical RAP paths used in this project:

```
Genotype prefixes (chromosome-split):  
/mnt/project/Bulk/Genotype Results/Genotype calls/ukb22418_c{1..22}_b0_v2  
  
Phenotype table:  
/mnt/project/Sool/GCTA_Heritability/data/phenotype/raw_blood_phenotypes.csv  
  
Covariate table:  
/mnt/project/Sool/covariates_processed_all.tsv  
  
Analysis outputs (example layout):  
/mnt/project/Sool/Analysis/GCTA_Heritability/<run_tag>/...
```

Important RAP path rule:

When passing inputs to dx run, use project-relative paths with project-...:/path and do not prepend the project name.

4A. Software Versions, Installation, And Provenance

Workflow runtime components:

- * GCTA via Docker image:
– 'quay.io/biocontainers/gcta:1.94.1--h9ee0642_0'
- * R for phenotype/covariate preparation.
- * Python 3 for '.hsq' summarization.
- * 'dx' CLI for RAP job submission.

Key wrapper behaviors:

* 'scripts/gcta_docker.sh' runs 'gcta64' in Docker and mounts current working directory (plus '/mnt/project' and '/home/dnanexus' when present).

* 'scripts/run_gcta_make_grm.sh' wraps '--make-grm' with '--bfile' or '--mbfile' and optional '--make-grm-part'.

* 'scripts/run_gcta_greml_one_trait.sh' conditionally passes '--covar' and '--qcovar' only when files include at least one covariate column beyond 'FID IID'.

5. Detailed Input File Inventory

Genotype inputs

One of:

- * Single merged PLINK prefix ('--bfile'), or
- * Multi-prefix file ('--mbfile') listing one prefix per chromosome.

Expected files per prefix:

```
<prefix>.bed  
<prefix>.bim  
<prefix>.fam
```

Phenotype input

A CSV/TSV-style file with participant ID plus trait columns. Default expected trait naming pattern in prep script:

```
participant.p{code}_i0
```

for trait codes such as '30000', '30010', etc.

Covariate input

A CSV/TSV-style file with participant ID and covariate fields. By default this is separate from the phenotype file, but the prep script can reuse the phenotype file when '--covar-csv' is not provided.

Trait list

Default trait set is hardcoded in 'prepare_gcta_inputs.R' and also stored in:

```
traits/blood_trait_codes.txt
```

Missing-value tokens

By default, the prep script treats the following as missing:

```
"", NA, NaN, nan, NULL, null, -9
```

6. Main Workflow Logic

Step 1: Prepare trait-specific GCTA files

Script: 'scripts/prepare_gcta_inputs.R'

Core logic:

1. Parse required inputs ('--fam', '--pheno-csv') and optional covariate args.
2. Read '.fam' and use 'IID' by default ('--fam-id-column' can switch to 'FID').
3. Auto-detect phenotype/covariate delimiter from ',', tab, ';', or '|'.
4. Resolve ID columns and trait columns.
5. Select covariates using either explicit lists or preset-based discovery:
 - 'auto' - 'ukb_greml_common' - 'ukb_blood_lab_shared'
6. Align phenotype and covariates to FAM sample order.
7. Drop rows with missing phenotype or missing covariates.
8. Write per-trait files:
 - '<trait>.phen' - '<trait>.covar' - '<trait>.qcovar' - '<trait>.keep'
9. Write run-level manifests:
 - 'trait_manifest.tsv' - 'traits_to_run.txt' - 'covariate_selection.txt'

Important behavior:

ID normalization strips trailing ".0" from numeric-looking IDs and de-duplicates by first occurrence.

Step 2: Build GRM

Script: 'scripts/run_gcta_make_grm.sh'

Core GCTA call pattern:

```
gcta64 --make-grm --bfile|--mbfile ... [--keep ...] [--make-grm-part i n]
```

At UKB scale, full dense GRM construction can be memory-heavy. The repo supports:

- * Direct GRM on a sample subset using '--keep' (recommended for practical GREML).
- * Partitioned GRM construction with '--make-grm-part'.

Step 3: Run GREML

Scripts:

- * 'scripts/run_gcta_greml_one_trait.sh'
- * 'scripts/run_gcta_greml_all_traits.sh'

One-trait command behavior:

- * Requires '--grm' and '--trait'.
- * Expects trait directory structure:
 - 'gcta_inputs/traits/<trait>/<trait>.phen' - 'gcta_inputs/traits/<trait>/<trait>.covar' - 'gcta_inputs/traits/<trait>/<trait>.qcovar' - 'gcta_inputs/traits/<trait>/<trait>.keep'
- * Builds output prefix 'results/trait_<trait>'.
- * Runs GCTA with:

```
--reml
--grm
--pheno
--keep
--thread-num
--out
```

- * Adds '--covar' and/or '--qcovar' only if those files have at least 3 columns.
- * Optionally adds '--prevalence' for case-control traits.

Step 4: Summarize '.hsq' files

Script: 'scripts/summarize_hsq.py'

It scans 'trait_*.hsq' files and writes one TSV containing:

- * 'trait', 'n'
- * 'h2', 'h2_SE'
- * 'V(G)', 'V(G)_SE'
- * 'V(e)', 'V(e)_SE'
- * 'Vp', 'Vp_SE'
- * 'LRT', 'LRT_P'
- * 'logL', 'logL0'

7. Parameter Table And Meaning

Input preparation parameters

- * '--fam': PLINK FAM file for sample universe and FID/IID.
- * '--pheno-csv': phenotype table containing blood traits.
- * '--covar-csv': covariate table.
- * '--trait-codes': which trait codes to process.
- * '--trait-column-template': expected trait column naming convention.
- * '--covar-preset': covariate selection strategy ('auto', 'ukb_greml_common', 'ukb_blood_lab_shared').
- * '--num-pcs': number of PCs to include when discoverable.
- * '--add-age-squared': append age squared as extra qcovariate.

GRM parameters

- * '--bfile' vs '--mbfile': single-prefix or multi-prefix genotype input.
- * '--keep': optional sample subset restriction.
- * '--make-grm-part i n': partitioned GRM build mode.
- * '--threads': thread count for GCTA.

GREML parameters

- * '--grm': GRM prefix.
- * '--trait': trait code for a one-trait run.
- * '--prevalence': optional liability-scale argument for case-control traits.
- * '--threads': GCTA thread count.

8. How To Run The Workflow

Option A: Launch RAP Step 1 and Step 2 with provided submitters

Step 1 prep submission:

```
Rscript scripts/submit_step1_prepare_dx.R --yes
```

or

```
scripts/submit_step1_prepare_dx.sh --yes
```

Step 2 GRM submission:

```
Rscript scripts/submit_step2_grm_dx.R --yes
```

These launchers construct 'dx run app-swiss-army-knife' calls with sensible repo defaults and RAP project discovery via 'dx pwd'.

Option B: Manual command flow

1. Prepare inputs:

```
Rscript scripts/prepare_gcta_inputs.R ...
```

2. Build GRM:

```
bash scripts/run_gcta_make_grm.sh ...
```

3. Run all traits:

```
bash scripts/run_gcta_greml_all_traits.sh ...
```

4. Summarize outputs:

```
python3 scripts/summarize_hsq.py ...
```

Option C: RAP Swiss Army Knife jobs for heavy compute

Use 'dx run app-swiss-army-knife' with explicit '-iin' inputs and '-icmd' commands for GRM and GREML jobs.

For long GREML runs, one trait per job is preferred for easier reruns and clearer fault isolation.

9. Expected Output Directory Layout

```
gcta_inputs/  
  covariate_selection.txt  
  trait_manifest.tsv  
  traits_to_run.txt  
  traits/  
    30000/  
      30000.phen  
      30000.covar  
      30000.qcovar  
      30000.keep  
  ...  
grm/  
  blood_grm.grm.bin  
  blood_grm.grm.N.bin  
  blood_grm.grm.id  
  blood_grm.log  
results/  
  trait_30000.hsq  
  trait_30000.log  
  trait_30010.hsq  
  ...  
  hsq_summary.tsv
```

When using partitioned GRM runs, additional 'part_*' GRM artifacts appear and must be merged before downstream use.

10. Quality Checks After Running

Input preparation checks:

```
head gcta_inputs/trait_manifest.tsv  
head gcta_inputs/covariate_selection.txt  
wc -l gcta_inputs/traits_to_run.txt
```

Trait file checks:

```
for t in $(cat gcta_inputs/traits_to_run.txt); do  
  wc -l gcta_inputs/traits/$t/$t.phen gcta_inputs/traits/$t/$t.covar gcta_inputs/traits/$t/$t.qcovar gcta_inputs/traits/$t/$t.keep  
done
```

GRM checks:

```
ls -lh grm/blood_grm.grm.bin grm/blood_grm.grm.N.bin grm/blood_grm.grm.id
```

GREML checks:

```
ls -l results/trait_*.hsq | wc -l  
head results/hsq_summary.tsv
```

Sanity expectations:

- * Number of '.hsq' files should match analyzable trait count.
- * 'hsq_summary.tsv' should include non-empty 'n', 'h2', and 'h2_SE' for passing runs.
- * Traits with 'n=0' after prep should not be run and should be visible in 'trait_manifest.tsv'.

11. Failure Modes And How To Debug

Missing or malformed trait files

Symptom:

```
Missing required file: gcta_inputs/traits/<trait>/<trait>.phen
```

Actions:

- * Rerun 'prepare_gcta_inputs.R'.
- * Check trait code exists in phenotype table columns.
- * Check 'trait_manifest.tsv' and warnings about missing traits.

Covariates silently excluded

Symptom:

```
Notice: skipping --covar because file has <3 columns
```

Actions:

- * Inspect '<trait>.covar'/'<trait>.qcovar' column counts.
- * Revisit '--covar-preset' or explicit '--covar-cols/--qcovar-cols'.

GRM memory failure at UKB scale

Symptom:

- * GCTA reports very high memory requirements or job termination.

Actions:

- * Build GRM on a subset via '--keep'.
- * Use '--make-grm-part' and merge artifacts when full GRM construction is required.
- * Increase RAP instance size for GRM jobs.

RAP path or permissions errors

Symptom:

* 'dx run' cannot find inputs.

Actions:

- * Confirm 'PROJECT_ID=\$(dx pwd | sed 's/:.*//)'.
* Use project-relative 'project-.../path' inputs.
* Confirm object presence with 'dx find data --path ... --name ...'.

No '.hsq' outputs for some traits

Actions:

- * Open 'results/trait_<trait>.log'.
- * Verify per-trait keep list has sufficient samples.
- * Confirm GRM and phenotype sample IDs align.

12. Known Caveats

- * Full dense UKB GRM construction and full-sample GREML are often impractical in standard RAP job sizes.
- * Results depend strongly on sample subset definition ('--keep'), covariate specification, and phenotype missingness filtering.
- * 'prepare_gcta_inputs.R' de-duplicates IDs by first occurrence; upstream table duplicates should be audited if they are biologically meaningful.
- * This repo is configured for continuous blood traits. Liability-scale interpretation needs additional care for binary traits and requires '--prevalence'.

13. Minimal Handoff Checklist

Before handing off outputs to another analyst:

1. Save exact Step 1/Step 2/Step 3/Step 4 command lines.
2. Save GRM prefix and whether '--keep' was used.
3. Save 'gcta_inputs/covariate_selection.txt' and 'trait_manifest.tsv'.
4. Save all 'results/trait_*.hsq' and 'results/hsq_summary.tsv'.
5. Record GCTA image tag: '1.94.1--h9ee0642_0'.
6. Record RAP project path and destination folders used for jobs.

14. How To Run This Again With New Trait Sets

1. Choose new trait codes and confirm their phenotype column names.
2. Run 'prepare_gcta_inputs.R' with updated '--trait-codes' and, if needed, '--trait-column-template'.
3. Check 'trait_manifest.tsv' for non-zero analyzable sample counts.
4. Rebuild or reuse a compatible GRM built from the intended sample universe.
5. Run GREML per trait and regenerate 'hsq_summary.tsv'.
6. Compare new and old summaries with explicit notes on sample universe, covariates, and trait definitions.

15. Source List

- * Repository README and run commands:
– 'README.md'

* RAP runbook:

– ‘dnanexus/runbook_swiss_army_knife.md’

* Core workflow scripts:

– ‘scripts/prepare_gcta_inputs.R’ – ‘scripts/run_gcta_make_grm.sh’ –
‘scripts/run_gcta_greml_one_trait.sh’ – ‘scripts/run_gcta_greml_all_traits.sh’ –
‘scripts/summarize_hsq.py’ – ‘scripts/gcta_docker.sh’ – ‘scripts/submit_step1_prepare_dx.R’ –
‘scripts/submit_step1_prepare_dx.sh’ – ‘scripts/submit_step2_grm_dx.R’

* Trait list:

– ‘traits/blood_trait_codes.txt’